Projektarbeit

Fachbereich M – Master Maschinenbau Mechatronik | WS 2025/2026

Entwicklung, Integration und Validierung einer ROS2-basierten Hinderniserkennung mit Reaktionsstrategie für automatisierte Modellfahrzeuge

Autoren:	Aly Elbermawy (5335848) Omar Elalami (5379642) Zakaria Elaboudi (5223183)
Betreuer:	Prof. Dr. Benedikt Lattke
Abgabedatum:	12. April 2026

Kurzfassung

Im Rahmen dieser Projektarbeit wurde ein scan-basierter *Follow-the-Gap*-Stack (FTG) für das MXCarkit entwickelt, in die bestehende ROS 2-Fahrzeugarchitektur integriert und experimentell validiert. Das Fahrzeug basiert auf einer Jetson-Plattform, verwendet einen 2D-LiDAR als primäre Sensorik für die lokale Umfelderfassung und wird über eine Ackermann-Steuerkette mit VESC angesteuert. Ziel der Arbeit war es, einen robusten, modularen und sicher testbaren Datenpfad von `/scan` bis zur autonomen Ausgabe `/autonomous/ackermann_cmd` bereitzustellen und dessen Funktionsfähigkeit unter realen Randbedingungen nachzuweisen.

Die realisierte Hauptarchitektur besteht aus vier funktional getrennten Stufen: einer LiDAR-Vorverarbeitung mit TF-basierter Rezentrierung und frontaler Sichtfeldselektion, dem reaktiven C++-Kern `follow_the_gap_v0` zur Bestimmung einer befahrbaren Lücke, einer Planner-Stufe zur Begrenzung des Lenkwinkels und zur adaptiven Geschwindigkeitswahl sowie einer Control-Stufe zur Überführung der Sollwerte in Ackermann-kompatible Fahrbefehle. Damit wurde eine klare Trennung zwischen Wahrnehmung, lokaler Richtungswahl, fahrdynamischer Policy und fahrzeugspezifischer Schnittstelle erreicht. Der frühere Legacy-Pfad über `obstacle_substitution` bleibt aus Kompatibilitätsgründen im Repository erhalten, stellt jedoch nicht mehr den primären Laufzeitpfad dar.

Die theoretische Grundlage bildet das Follow-the-Gap-Verfahren nach Sezer und Gokasan, ergänzt durch die CTU-/F1TENTH-Referenzimplementierung und deren Übertragung in eine moderne ROS 2-Umgebung. Auf dieser Basis wird gezeigt, wie ein klassischer lokaler reaktiver Planungsansatz in eine aktuelle modulare Fahrzeugsoftware integriert werden kann, ohne die bestehende Sicherheits- und Steuerlogik des MXCarkit zu umgehen.

Die experimentelle Validierung erfolgte anhand realer Fahrversuche in strukturierten Innenraumszenarien mit statischen Hindernissen und Eckdurchfahrt. Zur Analyse wurden alle relevanten Systemgrößen mit `rosbag2` aufgezeichnet und sowohl online mit Foxglove als auch offline über Zeitreihenplots und Kenngrößen ausgewertet. Die Ergebnisse zeigen, dass das System Hindernisse grundsätzlich zuverlässig als freie bzw. belegte Bereiche im LiDAR-Scan interpretieren, kollisionsfreie Ausweichmanöver erzeugen und in die bestehende Fahrzeugsteuerung integrieren kann. Gleichzeitig wurde deut-

lich, dass die Qualität des Fahrverhaltens wesentlich von der Parametrierung abhängt, insbesondere von Sichtfeld, Clearance-Schwellen, Steering-Slowdown und den internen geometrischen FTG-Konstanten. Die Arbeit liefert damit nicht nur einen funktionsfähigen Stack, sondern auch eine systematische Bewertung seiner Leistungsfähigkeit, Robustheit und Grenzen.

Inhaltsverzeichnis

Kurzfassung **1**

Inhaltsverzeichnis **III**

Abbildungsverzeichnis **V**

Tabellenverzeichnis **VI**

1 Grundlagen **1**

1.1 Aufgabenstellung und Ziel der Arbeit 1

1.2 Das MXCarkit als Zielplattform 1

1.3 Grundlagen von ROS 2 3

1.3.1 Workspace 3

1.3.2 Paket 4

1.3.3 Node 4

1.3.4 Launch-Dateien und Parameter 4

1.4 Stand der Technik 5

1.4.1 Vergleich bestehender Planungsansätze 5

1.4.2 Sensorik zur Umfeldwahrnehmung 5

1.5 Follow-the-Gap als reaktiver Planungsansatz 6

1.5.1 Grundprinzip und methodisches Vorgehen 6

1.5.2 Eigenschaften des Verfahrens 7

1.5.3 Scan-basierte Umsetzung im ROS 2-Kontext 7

2 Systembeschreibung und Durchführung **8**

2.1 Gesamtarchitektur des Systems 8

2.2 Algorithmische Grundlage 9

2.3 Stufe 1: LiDAR-Vorverarbeitung 10

2.3.1 Node-Detailbeschreibung der Perception-Stufe 10

2.3.2 Rolle der Perception-Stufe im Gesamtsystem 12

2.4 Stufe 2: Follow-the-Gap-Kern 12

2.4.1 Verarbeitungsschritte 12

2.4.2 Node-Detailbeschreibung des FTG-Kerns 13
2.4.3 Interne geometrische Modellannahmen 13
2.5 Stufe 3: Planner 13
2.5.1 Funktionale Rolle 14
2.5.2 Node-Detailbeschreibung des Planners 14
2.5.3 Interpretation der Sensordaten 15
2.6 Stufe 4: Control 15
2.6.1 Aufgabe 15
2.6.2 Node-Detailbeschreibung der Control-Stufe 15
2.7 Versuchsdurchführung 16
2.7.1 Versuchsaufbau 16
2.7.2 Durchführung der Testläufe 17
2.7.3 Parametrierung des Systems 17
2.7.4 Messmethodik und Datenerfassung 20

3 Diskussion und Bewertung 22
3.1 Bewertung der gewählten Gesamtarchitektur 22
3.2 Analyse der Planner- und Control-Schicht 22
3.3 Bewertung der experimentellen Ergebnisse 23
3.3.1 Quantitativer Vergleich der Validierungsläufe 23
3.3.2 Zeitreihenbasierte Detailanalyse 24
3.3.3 Bewertung der Hinderniserkennung 26
3.3.4 Bewertung der Ausweichmanöver 27
3.3.5 Einfluss der Parametrierung auf das Fahrverhalten 28
3.3.6 Empfehlungen aus der Validierung 28
3.3.7 Grenzfälle und Grenzen des Systems 29

4 Fazit und Ausblick 30
4.1 Fazit 30
4.2 Ausblick 31
Schlussbemerkung 32

Literaturverzeichnis 33

Abbildungsverzeichnis

1.1 Draufsicht des MXCarkit mit Markierung der Hauptkomponenten: NVIDIA Jetson, LiDAR und Tiefenkamera. 2
1.2 Systemarchitektur und Hardware-Komponenten des MXCarkit [8]. . . . 3

2.1 Gesamtarchitektur des scan-basierten FTG-Stacks. Die Pipeline reicht vom Rohscan bis zur Ackermann-Ausgabe und zeigt die funktionale Trennung zwischen Wahrnehmung, Planung und Steuerung. [7] 9
2.2 Schematische Darstellung des Follow-the-Gap-Prinzips. [6] 10
2.3 Validierungsstrecke Test 1: L-förmiger Korridor mit asymmetrischer Hindernisfolge und Eckrestriktion. 16
2.4 Validierungsstrecke Test 2: Spiegelbildliche Hindernisanordnung zur Untersuchung der Robustheit gegenüber variierter lateraler Objektreihenfolge. 16
2.5 Exemplarisches Foxglove-Layout zur Online-Beobachtung des FTG-Stacks während der Validierungsfahrten. Dargestellt sind Sensorik, Visualisierungstopics und zentrale Zustandsgrößen des Planungs- und Steuerpfads. 21

3.1 Zeitreihenbasierte Auswertung von Validierungslauf Test 1. Dargestellt sind Frontabstand, Heading- und Gap-Winkel, Sollgeschwindigkeit sowie Zustandsgrößen des Planners. 25
3.2 Zeitreihenbasierte Auswertung von Validierungslauf Test 2. Dargestellt sind Frontabstand, Heading- und Gap-Winkel, Sollgeschwindigkeit sowie Zustandsgrößen des Planners. 26

Tabellenverzeichnis

2.1 Verwendete Laufzeitparameter des FTG-Stacks während der Validierungsfahrten. 18
2.2 Interne FTG-Konstanten des C++-Kerns `follow_the_gap_v0`. 20

3.1 Vergleich zentraler Metriken der beiden Validierungsläufe. 24

1 Grundlagen

Dieses Kapitel legt die fachlichen und systemtechnischen Grundlagen der vorliegenden Arbeit dar. Zunächst wird die Aufgabenstellung präzise eingeordnet und das Zielsystem beschrieben. Anschließend werden die relevanten Konzepte von ROS 2 erläutert, die für das Verständnis der implementierten Architektur notwendig sind. Abschließend wird der Follow-the-Gap-Algorithmus sowohl in seiner ursprünglichen Form als auch in der für diese Arbeit relevanten scan-basierten Ausprägung dargestellt.

1.1 Aufgabenstellung und Ziel der Arbeit

Im Rahmen dieser Arbeit wird eine Fahrfunktion zur Hinderniserkennung für ein automatisiertes Modellfahrzeug auf Basis des MxCarkit-Systems entwickelt. Ziel ist es, eine robuste und echtzeitfähige Lösung zu konzipieren, die Hindernisse im Fahrweg zuverlässig erkennt und darauf mit geeigneten Reaktionsstrategien wie Abbremsen oder Ausweichen reagiert.

Die Aufgabenstellung umfasst die Entwicklung, Integration und Validierung einer ROS 2-basierten Softwarearchitektur, welche Sensordaten, insbesondere von LiDAR- oder Kamerasystemen, verarbeitet und daraus sichere Fahrentscheidungen ableitet. Dabei sollen sowohl geeignete Verfahren zur Umfeldwahrnehmung als auch eine darauf abgestimmte Reaktionslogik implementiert und experimentell untersucht werden.

Ein weiterer Schwerpunkt liegt auf der Systemintegration in die bestehende Fahrzeugarchitektur sowie der experimentellen Validierung unter Laborbedingungen. Abschließend erfolgt eine Bewertung der Leistungsfähigkeit, Robustheit und Grenzen des entwickelten Systems, um dessen Einsatzpotenzial für autonome Fahrfunktionen einzuordnen.

1.2 Das MXCarkit als Zielplattform

Das MXCarkit (MXCK) stellt die physische und softwareseitige Zielplattform dieser Arbeit dar. Es handelt sich um ein Jetson-basiertes Versuchsfahrzeug, das für Lehre und Entwicklung im Bereich autonomer Fahrfunktionen konzipiert ist.

Abbildung 1.1 zeigt den physischen Aufbau des Versuchsfahrzeugs mit den für diese Arbeit zentralen Sensorkomponenten.

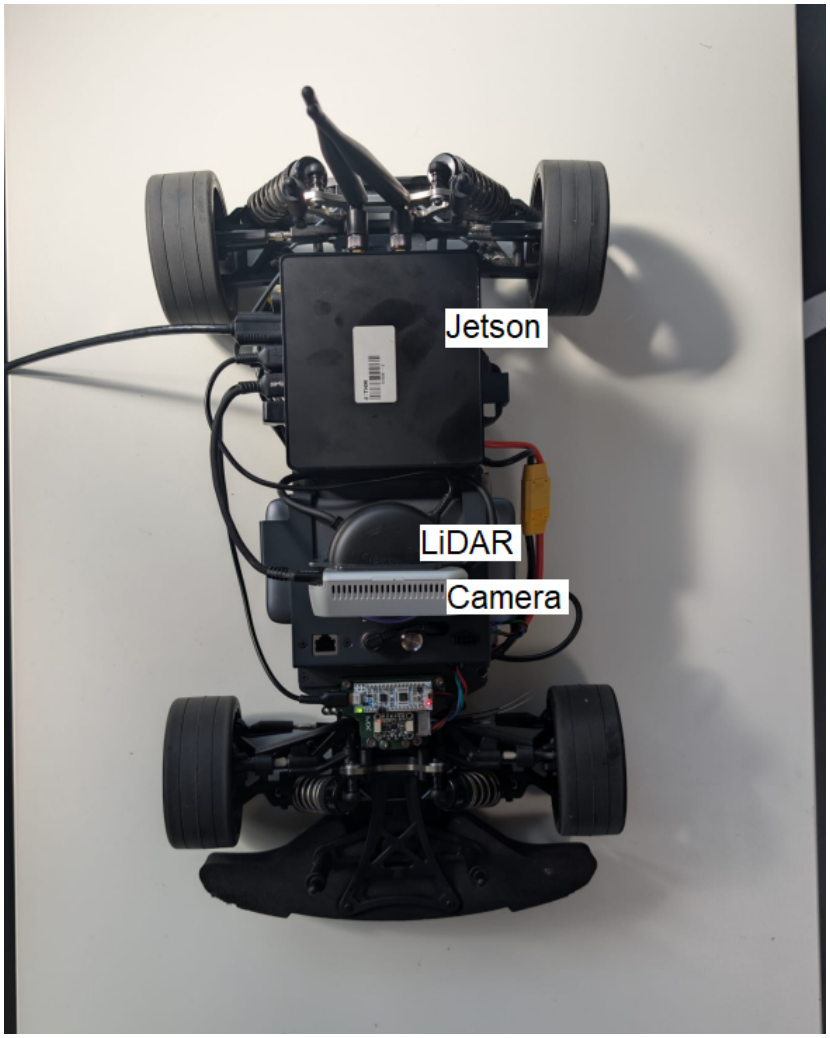

Abbildung 1.1: Draufsicht des MXCarkit mit Markierung der Hauptkomponenten: NVIDIA Jetson, LiDAR und Tiefenkamera.

Die Systemarchitektur des MXCK ist modular aufgebaut und nutzt eine containerisierte Umgebung. Eine detaillierte Übersicht der Hardware-Komponenten und deren Vernetzung ist in Abbildung 1.2 dargestellt.

Abbildung 1.2: Systemarchitektur und Hardware-Komponenten des MXCarkit [8].

Zentrale Funktionalitäten sind in separaten Workspaces organisiert, insbesondere:

- `mxck2_ws` für die Kernsysteme (Sensorik, Steuerung),
- `development_ws` für experimentelle Erweiterungen.

1.3 Grundlagen von ROS 2

Da die gesamte Systemarchitektur auf ROS 2 basiert, ist ein präzises Verständnis der zugrunde liegenden Konzepte erforderlich.

1.3.1 Workspace

Ein ROS 2-Workspace ist ein Verzeichnis, in dem mehrere Pakete gemeinsam verwaltet und gebaut werden. Die typische Struktur umfasst die Verzeichnisse `src/`, `build/`, `install/` und `log/`. Der Build-Prozess erfolgt üblicherweise über `colcon`.

Im Kontext des MXCK wird diese Struktur direkt genutzt, wobei unterschiedliche Workspaces verschiedene Verantwortlichkeiten abdecken.

1.3.2 Paket

Ein Paket stellt die zentrale organisatorische Einheit in ROS 2 dar. Es bündelt eine klar abgegrenzte Funktionalität, beispielsweise im Bereich Perception, Planung oder Steuerung.

Je nach Implementierungssprache unterscheiden sich die notwendigen Dateien:

- Python: `package.xml`, `setup.py`, `setup.cfg`,
- C++: `package.xml`, `CMakeLists.txt`.

Zusätzlich können Launch-Dateien und YAML-Konfigurationen Bestandteil eines Pakets sein.

1.3.3 Node

Ein Node ist ein eigenständiger Prozess innerhalb des ROS 2-Systems. Nodes kommunizieren über Topics, Services oder Actions und bilden gemeinsam einen verteilten Verarbeitungsgraphen.

Im vorliegenden System entspricht jede funktionale Stufe des FTG-Stacks einem eigenen Node, was eine klare Trennung der Verantwortlichkeiten ermöglicht.

1.3.4 Launch-Dateien und Parameter

ROS 2 verwendet bevorzugt Python-basierte Launch-Dateien zur Orchestrierung mehrerer Nodes. Diese definieren:

- welche Nodes gestartet werden,
- welche Parameter geladen werden,
- sowie die verwendeten Topic-Namen und Namespaces.

YAML-Dateien dienen dabei der strukturierten Parametrierung der Nodes. Für die Reproduzierbarkeit des Systems ist die Konsistenz zwischen Launch-Dateien, Parametern und tatsächlicher Node-Konfiguration entscheidend.

1.4 Stand der Technik

Die autonome Navigation von Robotern wird in der Fachliteratur klassisch in die globale Pfadplanung und die lokale Hindernisvermeidung unterteilt. Während statische Planer eine vollständige Karte der Umgebung benötigen, agiert die Hindernisvermeidung rein reaktiv auf Basis aktueller Sensordaten.

1.4.1 Vergleich bestehender Planungsansätze

In der Entwicklung haben sich verschiedene Ansätze zur lokalen Navigation etabliert, die jedoch spezifische systemtechnische Nachteile aufweisen:

- **Bug-Algorithmen**: Diese folgen dem Prinzip, sich direkt auf ein Ziel zuzubewegen und bei Kontakt ein Hindernis am Rand zu umfahren. Dies führt jedoch oft zu ineffizienten Trajektorien und einer gefährlich nahen Annäherung an Objekte.
- **Artificial Potential Fields (APF)**: Hierbei werden Hindernisse als abstoßende und das Ziel als anziehende Potentiale modelliert. Ein kritisches Problem stellt das Auftreten lokaler Minima dar, bei denen sich die wirkenden Kräfte gegenseitig aufheben und der Roboter vor Erreichen des Ziels verharrt [1].
- **Vector Field Histogram (VFH)**: Dieser Ansatz nutzt ein polares Histogramm zur Richtungsbestimmung. Er ist zwar zielorientiert, vernachlässigt jedoch häufig die spezifischen kinematischen Beschränkungen des Fahrzeugs, was zu nicht fahrbaren Pfaden führen kann.

Aufgrund dieser Defizite wurde für die vorliegende Arbeit die *Follow-the-Gap Method* (FGM) gewählt. Im Gegensatz zu APF-Methoden ist FGM mathematisch resistent gegen lokale Minima, da der Algorithmus gezielt nach dem größten freien Bereich sucht, anstatt lediglich Kraftvektoren zu summieren [1]. Zudem integriert FGM die kinematischen Beschränkungen (Non-Holonomie) des Fahrzeugs direkt in den Auswahlprozess der Lücke, was eine höhere Trajektoriengüte und Sicherheit gewährleistet.

1.4.2 Sensorik zur Umfeldwahrnehmung

Für die Umsetzung einer robusten Hindernisvermeidung ist die Wahl der Sensorik entscheidend. In dieser Arbeit wird ein 2D-LiDAR-System (Light Detection and Ranging) als Primärsensor eingesetzt. Die Entscheidung für LiDAR gegenüber kamerabasierten Systemen (Vision) basiert auf folgenden wissenschaftlichen Kriterien:

1. **Präzision und Reichweite**: LiDAR liefert direkte und hochpräzise Distanzmessungen mit einer geringen Latenz, was für reaktive Algorithmen wie FGM essentiell ist [1].

2. **Recheneffizienz**: Im Vergleich zur komplexen Bildverarbeitung (Computer Vision), die hohe Anforderungen an die CPU/GPU stellt, erlauben LiDAR-Scans eine schnelle Extraktion von Hindernissen in Form von Distanz-Arrays.

3. **Beleuchtungsunabhängigkeit**: LiDAR-Systeme arbeiten als aktive Sensoren unabhängig von externen Lichtverhältnissen, was die Robustheit der Hinderniserkennung in unterschiedlichen Laborumgebungen sicherstellt.

Durch die Kombination der LiDAR-basierten Distanzmessung mit der geometrischen Logik des Follow-the-Gap-Verfahrens wird eine Architektur realisiert, die sowohl die Anforderungen an die Echtzeitfähigkeit als auch an die Kollisionssicherheit erfüllt.

1.5 Follow-the-Gap als reaktiver Planungsansatz

Der Follow-the-Gap-Algorithmus (FGM) ist ein lokaler Planungsansatz, der eine sichere Durchquerung von Umgebungen ermöglicht, ohne auf eine globale Karte angewiesen zu sein [1]. Er berechnet basierend auf der aktuellen Sensordatenlage reaktiv den nächsten Bewegungsschritt [1].

1.5.1 Grundprinzip und methodisches Vorgehen

Die Implementierung nach Sezer und Gokasan transformiert das Problem zunächst durch eine Hindernisvergrößerung (*Obstacle Enlargement*) in einen Punkt-Roboter-Ansatz [1]. Dabei wird der Radius des Fahrzeugs rechnerisch zum Radius der Hindernisse addiert, um eine kollisionsfreie Trajektorie innerhalb der berechneten Lücken zu garantieren [1].

Der Algorithmus folgt drei wesentlichen Schritten [1]:

1. **Berechnung des Gap-Arrays**: Basierend auf den Sensordaten sowie den Sichtfeldgrenzen (*Field of View*) und den kinematischen Grenzen (*Nonholonomic Constraints*) werden freie Bereiche zwischen den Hindernissen identifiziert [1].

2. **Bestimmung des Gap-Zentrums**: Statt lediglich den arithmetischen Mittelwert der Winkel zu bilden, nutzt das Verfahren geometrische Ansätze wie das Cosinus- und das Apollonius-Theorem, um den Median-Vektor zur Mitte der Lücke zu berechnen [1]. Dies sorgt für eine Trajektorie, die den Abstand zu den begrenzenden Objekten maximiert und somit die Sicherheit erhöht [1].

3. **Berechnung des finalen Kurswinkels**: Eine Fusing-Funktion kombiniert den berechneten Winkel zum Gap-Zentrum mit dem Winkel zum globalen Ziel [1].

Das System gewichtet dabei die Sicherheit (Gap) höher, wenn Hindernisse in unmittelbarer Nähe zum Fahrzeug detektiert werden, während bei freien Fahrwegen die Zielerreichung priorisiert wird [1].

1.5.2 Eigenschaften des Verfahrens

Das FGM-Verfahren zeichnet sich durch folgende Merkmale aus:

- **Minimale Parametrierung**: Die Steuerung des Verhältnisses zwischen Zielorientierung und Sicherheit erfolgt über lediglich einen zentralen Tuning-Parameter (α) [1].
- **Berücksichtigung der Kinematik**: Der Algorithmus bezieht den minimalen Wendekreis des Fahrzeugs direkt in die Lückenberechnung ein, wodurch nur physikalisch fahrbare Trajektorien generiert werden [1].
- **Vermeidung lokaler Minima**: Im Gegensatz zu klassischen Potentialfeldmethoden weist FGM aufgrund seiner geometrischen Struktur kein inhärentes Problem mit lokalen Minima auf [1].

1.5.3 Scan-basierte Umsetzung im ROS 2-Kontext

In der praktischen Umsetzung für das MXCarkit wird dieser theoretische Ansatz in eine modulare ROS 2-Softwarearchitektur überführt [7]. Die primäre Eingangsgröße bildet dabei die Nachricht `sensor_msgs/LaserScan` des LiDAR-Sensors [7]. Die Implementierung im Repository `ftg_mxck` bildet die methodischen Schritte des Prozesses als reaktive Verarbeitungskette ab [7]:

```
/scan (Rohdaten des LiDAR)
  -> Vorverarbeitung (Filterung, Beruecksichtigung der
     Fahrzeuggeometrie)
  -> FGM-Kern (Geometrische Berechnung des Gap-Zentrums)
  -> Planner (Gewichtung von Sicherheit und Zielvorgabe via Alpha)
  -> Control (Ausgabe als /autonomous/ackermann_cmd)
```

Diese Struktur ermöglicht eine stabile Integration des algorithmischen Kerns von Sezer und Gokasan in ein verteiltes System und wahrt dabei die Echtzeitfähigkeit der Hindernisvermeidung [7].

2 Systembeschreibung und Durchführung

Dieses Kapitel beschreibt die konkret umgesetzte Systemarchitektur sowie die methodische Durchführung der Implementierung und Validierung des Follow-the-Gap-Stacks im Repository `ftg_mxck`. Im Mittelpunkt steht der scan-basierte Hauptpfad, der als primäre Laufzeitarchitektur des Systems definiert wurde. Ziel dieses Kapitels ist es, die funktionale Rolle der einzelnen Softwarestufen, deren Zusammenspiel innerhalb der ROS 2-Pipeline sowie die experimentelle Vorgehensweise bei der Validierung nachvollziehbar darzustellen.

2.1 Gesamtarchitektur des Systems

Die entwickelte Pipeline folgt einer strikt modularen Struktur, die sich in vier logisch getrennte Verarbeitungsebenen gliedert:

1. LiDAR-Vorverarbeitung (*Perception*)
2. reaktiver Planerkern (*Follow-the-Gap*)
3. policy-basierte Entscheidungslogik (*Planner*)
4. fahrzeugspezifische Schnittstelle (*Control*)

Die resultierende Datenflusskette ist in Abbildung 2.1 dargestellt. Die Architektur ist so ausgelegt, dass der entwickelte FTG-Stack nicht direkt in die Aktorik eingreift, sondern ausschließlich Sollwerte an die bestehende Fahrzeugsteuerung übergibt. Dadurch bleibt die vorhandene Sicherheits- und Moduslogik des MXCarkit unverändert erhalten, während die reaktive Hindernisvermeidung als zusätzlicher autonomer Entscheidungsweg integriert wird.

```
/scan (LaserScan, frame: laser)
  |
  ▼
┌──────────────────────┐
│  scan_preprocessor_node │  Paket: mxck_ftg_perception
│  TF: base_link ← laser  │  Rezentrierung auf Fahrzeug-Front
│  FOV: ±60° (120°)       │  Clipping: 0.25–10.0 m
└──────────────────────┘
         |
         |
         ├──▶ /autonomous/ftg/scan_filtered  (LaserScan, frame: base_link)
         └──▶ /autonomous/ftg/front_clearance (Float32, Meter)
               |                                  |
               ▼                                  |
┌──────────────────────┐                          |
│    follow_the_gap_v0    │  Paket: follow_the_gap_v0│
│    (C++, input_mode=scan)│  CTU FTG-Algorithmus     |
└──────────────────────┘                          |
         |                                        |
         |                                        |
         ├──▶ /final_heading_angle (Float32, rad)    |
         ├──▶ /gap_found           (Bool)            |
         ├──▶ /visualize_obstacles (Marker)          |
         ├──▶ /visualize_largest_gap (PointStamped)  |
         └──▶ /visualize_final_heading_angle (PoseStamped)
               |                                  |
               ▼                                  ▼
┌──────────────────────────────────────────────┐
│              ftg_planner_node                  |
│  Paket: mxck_ftg_planner                       |
│  Speed-Policy: clearance + steering → Geschwindigkeit|
│  Heading-Smoothing + Speed-Smoothing           |
└──────────────────────────────────────────────┘
         |
         |
         ├──▶ /autonomous/ftg/gap_angle      (Float32, rad)
         ├──▶ /autonomous/ftg/target_speed   (Float32, m/s)
         └──▶ /autonomous/ftg/planner_status (String)
               |
               ▼
┌──────────────────────┐
│    ftg_command_node     │  Paket: mxck_ftg_control
│    → AckermannDriveStamped
└──────────────────────┘
         |
         |
         ├──▶ /autonomous/ackermann_cmd       (AckermannDriveStamped)
         └──▶ /autonomous/ftg/control_status  (String)
               |
               ▼
         [vehicle_control]  (extern, nicht Teil dieses Stacks)
```

Abbildung 2.1: Gesamtarchitektur des scan-basierten FTG-Stacks. Die Pipeline reicht vom Rohscan bis zur Ackermann-Ausgabe und zeigt die funktionale Trennung zwischen Wahrnehmung, Planung und Steuerung. [7]

2.2 Algorithmische Grundlage

Der eingesetzte Planungsansatz basiert auf dem Follow-the-Gap-Verfahren nach Sezer und Gokasan. Die konkrete Implementierung orientiert sich an der F1TENTH-/CTU-Referenz und wurde als C++-Kern in den Stack integriert. Der Algorithmus arbeitet rein lokal und benötigt weder globale Karteninformation noch eine explizite semantische Objekterkennung.

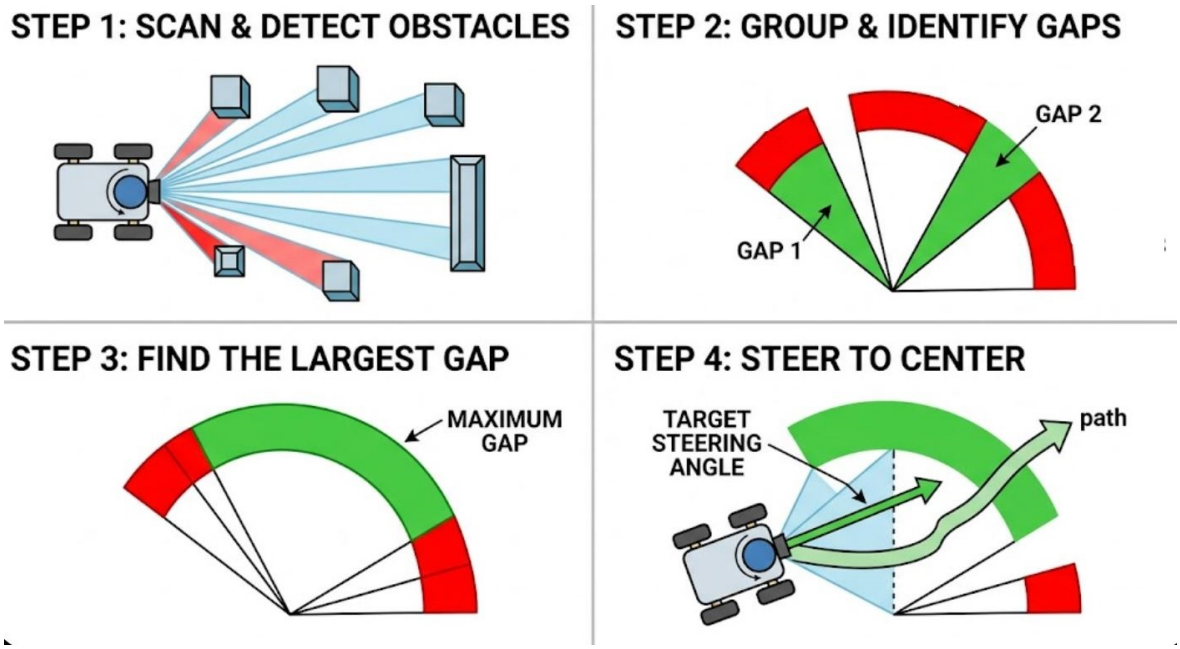

Abbildung 2.2: Schematische Darstellung des Follow-the-Gap-Prinzips. [6]

Die Grundidee besteht darin, aus einem eindimensionalen Distanzprofil des LiDARs freie und belegte Bereiche abzuleiten, die größte befahrbare Lücke zu bestimmen und daraus eine geeignete Zielrichtung abzuleiten. Das Verfahren ist damit als lokaler reaktiver Planer einzuordnen. Seine Stärke liegt in der geringen Latenz und der direkten Reaktion auf lokale Hindernisse; seine Grenze besteht darin, dass keine globale Pfadstrategie oder Umgebungsrepräsentation aufgebaut wird.

2.3 Stufe 1: LiDAR-Vorverarbeitung

Die erste Stufe transformiert den Rohscan in eine für den FTG-Kern geeignete Repräsentation. Ziel ist es, die geometrische Beziehung zwischen Sensor und Fahrzeug konsistent aufzulösen und aus dem vollständigen 360°-LiDAR-Signal einen für die lokale Fahrentscheidung relevanten Frontbereich abzuleiten.

2.3.1 Node-Detailbeschreibung der Perception-Stufe

Die Perception-Stufe besteht aus zwei Nodes: dem eigentlichen `scan_preprocessor_node` als produktiver Vorverarbeitungsstufe und dem optionalen Diagnose-Node `scan_front_window_check`. Beide Nodes arbeiten auf Basis des LiDAR-Topics `/scan` und der TF-Beziehung zwischen `laser` und `base_link`. Der aktuelle Repository-Stand beschreibt diese Trennung ebenfalls explizit.

`scan_preprocessor_node`

Der `scan_preprocessor_node` bildet die zentrale Wahrnehmungsstufe des Stacks. Er liest den Rohscan, bestimmt mit Hilfe von TF die Orientierung des LiDARs relativ zum Fahrzeugrahmen und berechnet daraus, welche Messstrahlen zur Fahrtrichtung gehören. Im aktuellen Code werden hierzu unter anderem ein `scan_topic`, ein `base_frame`, ein frontales Sichtfeld sowie Distanzgrenzen und optionale Glättungsparameter deklariert.

Funktional führt der Node folgende Schritte aus:

1. Einlesen des Rohscans von `/scan`.
2. TF-Abfrage der Transformation `base_link` ← `laser`.
3. Bestimmung der Fahrzeugvorwärtsrichtung im Scan unter Berücksichtigung der Sensorrotation.
4. Auswahl aller Messstrahlen innerhalb des definierten Frontfensters.
5. Clipping der Messwerte auf den zugelassenen Distanzbereich.
6. Optionale Glättung der Distanzwerte mittels gleitendem Mittelwert.
7. Publikation des rezentrierten Frontscans auf `/autonomous/ftg/scan_filtered`.
8. Publikation des minimalen Frontabstands auf `/autonomous/ftg/front_clearance`.
9. Ausgabe eines textuellen Diagnosezustands auf `/autonomous/ftg/status`.

Die wesentliche fachliche Funktion dieses Nodes besteht somit darin, die sensorische Umgebung in eine für den FTG-Kern geeignete, fahrzeugbezogene Darstellung zu überführen. Der resultierende Scan besitzt im `base_link`-Bezug eine direkte Interpretation der Fahrtrichtung, wodurch der nachfolgende Algorithmus nicht mehr auf die physische Sensormontage reagieren muss.

`scan_front_window_check`

Der optionale Diagnose-Node `scan_front_window_check` dient nicht der eigentlichen Fahrentscheidung, sondern der Plausibilitätsprüfung des frontalen Wahrnehmungsfensters. Er analysiert den Rohscan im selben geometrischen Bezug wie der Preprocessor und ermittelt den jeweils nächstgelegenen Punkt im Frontbereich. Zusätzlich können Marker veröffentlicht werden, um die Interpretation des Frontfensters in RViz oder Foxglove direkt sichtbar zu machen.

Im Rahmen dieser Arbeit wurde dieser Node primär als Diagnosewerkzeug genutzt, um sicherzustellen, dass die durch TF definierte Fahrzeugvorwärtsrichtung mit der tatsächlich ausgewerteten Scan-Geometrie übereinstimmt.

2.3.2 Rolle der Perception-Stufe im Gesamtsystem

Die Vorverarbeitung ist nicht bloß ein Vorfilter, sondern eine entscheidende geometrische Konsolidierungsstufe. Fehler in dieser Stufe — etwa ein falscher TF-Bezug, ein ungeeignetes Sichtfeld oder eine unpassende Clipping-Grenze — wirken sich unmittelbar auf die Gap-Erkennung und damit auf das gesamte Fahrverhalten aus. In der späteren Validierung zeigt sich genau dieser Zusammenhang: Parameter wie Sichtfeld und Clearance-Definition beeinflussen, ob das Fahrzeug eher flüssig oder konservativ reagiert.

2.4 Stufe 2: Follow-the-Gap-Kern

Der FTG-Kern bildet das algorithmische Zentrum des Systems. Er verarbeitet den vorbereiteten Frontscan und leitet daraus eine lokale Zielrichtung ab. Im aktuellen Stack ist dieser Kern als C++-Node `follow_the_gap` im Paket `follow_the_gap_v0` realisiert. Er kann zwar grundsätzlich sowohl auf Scan- als auch auf Hindernis-Inputs arbeiten, im hier betrachteten Hauptpfad wird er jedoch explizit im scan-basierten Modus betrieben.

2.4.1 Verarbeitungsschritte

Innerhalb des Algorithmus erfolgen im Wesentlichen:

1. Umwandlung des vorbereiteten Scans in eine interne Hindernisrepräsentation,
2. Identifikation von Hindernisbereichen,
3. Bestimmung zusammenhängender freier Bereiche (*Gaps*),
4. Auswahl der größten bzw. geeignetsten befahrbaren Lücke,
5. Berechnung eines Zielwinkels innerhalb dieser Lücke.

Die resultierende Zielrichtung ist ein rein geometrisches Ergebnis und enthält bewusst noch keine Aussage über Geschwindigkeit oder fahrzeugspezifische Dynamik.

2.4.2 Node-Detailbeschreibung des FTG-Kerns

Der Node `follow_the_gap` empfängt im scan-basierten Betrieb den Topic `/autonomous/ftg/scan_filtered`. Als primäre Ausgänge publiziert er:

- `/final_heading_angle` als kontinuierlichen Richtungswinkel,
- `/gap_found` als binäre Information, ob eine befahrbare Lücke detektiert wurde.

Zusätzlich publiziert der Node Visualisierungstopics, insbesondere:

- `/visualize_obstacles`,
- `/visualize_largest_gap`,
- `/visualize_final_heading_angle`.

Diese Visualisierungstopics wurden im Rahmen der Arbeit nicht als Teil der eigentlichen Steuerlogik, wohl aber als wichtige Hilfsmittel bei Debugging und Validierung verwendet. Sie ermöglichen, die vom FTG-Kern interpretierten Hindernispunkte, die gewählte größte Lücke sowie die resultierende Fahrtrichtung unmittelbar in einer 3D-Ansicht nachzuvollziehen. :contentReference[oaicite:6]index=6

2.4.3 Interne geometrische Modellannahmen

Ein wesentlicher Punkt des integrierten FTG-Kerns ist, dass die Fahrzeuggestalt nicht direkt als Breite oder Kontur parametrisiert wird, sondern indirekt über interne C++-Konstanten in die Gap-Bewertung eingeht. Besonders relevant ist hierbei `kCarRadius`, der zur Hindernis-Aufblähung genutzt wird. Dadurch wird algorithmisch entschieden, welche Lücken als passierbar und welche als blockiert interpretiert werden.

Aus wissenschaftlicher Sicht ist dies besonders wichtig, da die Bewertung der experimentellen Ergebnisse nicht allein aus den sichtbaren ROS-Parametern erklärt werden kann. Vielmehr wirken auch interne Geometrieannahmen auf das resultierende Verhalten, insbesondere in engen Korridoren und Corner-Situationen.

2.5 Stufe 3: Planner

Der Planner interpretiert die Ausgabe des FTG-Kerns im Kontext von Fahrzeugdynamik und Sicherheitsanforderungen. Im Gegensatz zum FTG-Kern handelt es sich hierbei nicht um einen Planungsalgorithmus im engeren Sinne, sondern um eine Entscheidungs- und Policy-Stufe, die aus einem reinen Richtungsvektor fahrbare Sollwerte ableitet.

2.5.1 Funktionale Rolle

Die Planner-Stufe übernimmt insbesondere folgende Aufgaben:

- Validierung der Eingangsdaten hinsichtlich Aktualität (*Freshness*),
- Begrenzung des resultierenden Lenkwinkels,
- adaptive Geschwindigkeitswahl in Abhängigkeit vom Frontabstand,
- textuelle Zustandsbewertung des aktuellen Planner-Zustands.

2.5.2 Node-Detailbeschreibung des Planners

Der `ftg_planner_node` liest drei zentrale Eingangsgrößen:

- den vom FTG-Kern gelieferten Heading-Winkel `/final_heading_angle`,
- das Gap-Erkennungssignal `/gap_found`,
- den vom Preprocessor berechneten Frontabstand `/autonomous/ftg/front_clearance`.

Auf dieser Basis erzeugt er drei zentrale Ausgangssignale:

- `/autonomous/ftg/gap_angle`,
- `/autonomous/ftg/target_speed`,
- `/autonomous/ftg/planner_status`.

Der Planner kombiniert damit die geometrische Richtungsauswahl des FTG-Kerns mit einer fahrdynamischen Sicherheitslogik. Die Geschwindigkeitswahl erfolgt über eine lineare Kombination zweier Einflüsse:

- eines Clearance-basierten Faktors in Abhängigkeit vom minimalen Frontabstand,
- eines Steering-basierten Faktors in Abhängigkeit von der Stärke des Ausweichwinkels.

Zusätzlich glättet der Planner sowohl den Heading-Winkel als auch den Geschwindigkeitsverlauf. Dadurch wird ein zu starkes Springen der Sollwerte zwischen aufeinanderfolgenden Scans reduziert, ohne die Möglichkeit zum schnellen Abbremsen in kritischen Situationen aufzugeben.

2.5.3 Interpretation der Sensordaten

Die Planner-Policy kann in vereinfachter Form wie folgt beschrieben werden:

- große Frontabstände → hohe Geschwindigkeit,
- kleine Frontabstände → reduzierte Geschwindigkeit,
- kritische Frontabstände → Stopp,
- große Ausweichwinkel → zusätzliche Geschwindigkeitsreduktion.

Diese Policy stellt sicher, dass der reaktive Charakter des FTG-Ansatzes nicht zu unsicheren Fahrzuständen führt, sondern in eine kontrollierte und physikalisch realisierbare Fahrstrategie überführt wird.

2.6 Stufe 4: Control

Die letzte Stufe des Systems übernimmt die Transformation der Planner-Ausgabe in fahrzeugspezifische Steuerbefehle.

2.6.1 Aufgabe

Die vom Planner berechneten Sollwerte für Winkel und Geschwindigkeit werden in ein Ackermann-kompatibles Nachrichtenformat überführt und an die bestehende Fahrzeugsteuerung übergeben. Dadurch entsteht die Schnittstelle zwischen dem autonomen FTG-Stack und der realen Aktorik des MXCarkit.

2.6.2 Node-Detailbeschreibung der Control-Stufe

Der `ftg_command_node` liest die beiden Planner-Ausgänge `/autonomous/ftg/gap_angle` und `/autonomous/ftg/target_speed`. Anschließend:

- prüft er die Aktualität der Eingangsdaten,
- skaliert bzw. invertiert den Lenkwinkel über `angle_to_steering_gain`,
- begrenzt den Lenkwinkel auf die physisch zulässige Obergrenze,
- begrenzt die Sollgeschwindigkeit auf den vorgegebenen Bereich,
- publiziert das Ergebnis als `/autonomous/ackermann_cmd`.

Zusätzlich publiziert der Node einen textuellen Diagnosezustand auf `/autonomous/ftg/control_status`. Diese Trennung zwischen funktionalem Ausgang und Diagnoseinformation ist für Debugging und Validierung besonders nützlich, da sich Sollwertbildung und tatsächlich weitergereichter Fahrbefehl getrennt analysieren lassen.

2.7 Versuchsdurchführung

Die Validierung des Systems erfolgte anhand realer Fahrversuche in einer strukturierten Innenraumumgebung. Ziel war es, die grundsätzliche Funktionsfähigkeit des scan-basierten Hindernisvermeidungsstacks experimentell nachzuweisen, den Einfluss zentraler Parameter auf das Fahrverhalten zu untersuchen und die Grenzen des Systems in typischen Engstellen- und Corner-Situationen sichtbar zu machen.

2.7.1 Versuchsaufbau

Die Testumgebung bestand aus einem L-förmigen Korridor mit definierten Breiten und gezielt platzierten statischen Hindernissen. Zur Untersuchung der Robustheit wurden zwei spiegelbildlich aufgebaute Validierungsstrecken verwendet, die sich in der lateralen Reihenfolge der Hindernisse unterschieden.

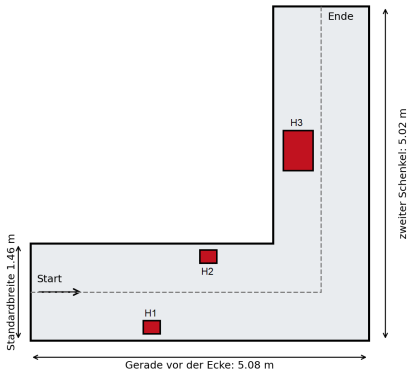

Abbildung 2.3: Validierungsstrecke Test 1: L-förmiger Korridor mit asymmetrischer Hindernisfolge und Eckrestriktion.

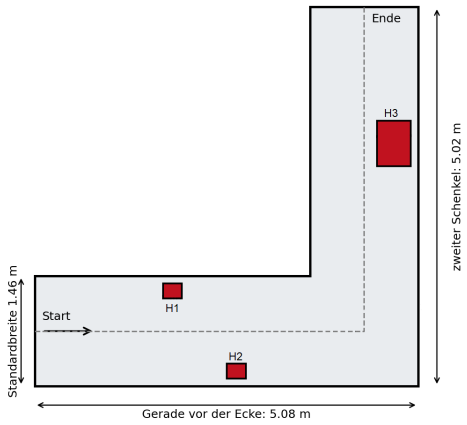

Abbildung 2.4: Validierungsstrecke Test 2: Spiegelbildliche Hindernisanordnung zur Untersuchung der Robustheit gegenüber variierter lateraler Objektreihenfolge.

Die gewählte Versuchsanordnung erlaubt die Untersuchung mehrerer für das System kritischer Situationen:

- freie Gerade mit stabiler Vorwärtsfahrt,
- asymmetrische Hindernisannäherung links bzw. rechts,
- Wahl zwischen unterschiedlich breiten Passagen,
- Eckdurchfahrt in einer 90°-Korridorstruktur.

2.7.2 Durchführung der Testläufe

Vor jedem Versuch wurden zunächst die sicherheitsrelevanten und plattformbezogenen Systemkomponenten gestartet. Dazu gehörten:

- die manuelle Fahrzeugsteuerung und Sicherheitslogik im `mxck2_control`-Container,
- die TF- und LiDAR-Bereitstellung,
- sowie der FTG-Stack im `mxck2_development`-Container.

Die Versuchsabfolge umfasste danach typischerweise:

1. Start der Vehicle-Control-Kette und Kalibrierung des Funksenders,
2. Start von TF und LiDAR,
3. Start einzelner FTG-Komponenten bzw. des vollständigen FTG-Stacks,
4. Plausibilitätsprüfung der zentralen Topics,
5. Durchführung der Fahrt im Modus *Autonomous*,
6. Aufzeichnung aller relevanten Topics mittels `rosbag2`.

2.7.3 Parametrierung des Systems

Die Validierungsfahrten wurden mit einer fest definierten Konfiguration des FTG-Stacks durchgeführt. Die Parametrierung ist für die Bewertung des Systemverhaltens von zentraler Bedeutung, da sie sowohl die geometrische Interpretation der Umgebung als auch die resultierende Fahrdynamik direkt beeinflusst. Insbesondere wirken sich Sichtfeld, Distanzgrenzen, Geschwindigkeitsgrenzen, Glättung und Abbremslogik unmittelbar auf das Ausweich- und Kurvenverhalten des Fahrzeugs aus.

Tabelle 2.1 fasst die im Validierungsbetrieb verwendeten Laufzeitparameter der drei zentralen Softwarestufen Perception, Planner und Control zusammen.

Tabelle 2.1: Verwendete Laufzeitparameter des FTG-Stacks während der Validierungsfahrten.

Node	**Parameter**	**Wert**	**Einfluss**
Perception	`front_center_deg`	0.0	Definiert Fahrtrichtung im `base_link`-Frame; TF berücksichtigt Montage.
Perception	`front_fov_deg`		Sichtfenster des FTG; größer = mehr Abdeckung, aber mehr Rand-Einfluss.
Perception	`clip_min_range_m`	0.25	Filtert sehr nahe Punkte und Eigenreflexionen.
Perception	`clip_max_range_m`	10.0	Begrenzt Auswertung auf relevanten Messbereich.
Perception	`enable_moving_average`	`true`	Glättet Distanzsignal zur Ausreißerreduktion.
Perception	`moving_average_window`	3	Bestimmt Glättungsstärke (größer = stabiler, aber träger).
Planner	`input_timeout_sec`	0.50	Stopp bei zu alten Eingangsdaten.
Planner	`max_abs_gap_angle_rad`	0.45	Begrenzung des maximalen Lenkwinkels.
Planner	`cruise_speed_mps`	0.45	Maximale Geschwindigkeit in freier Fahrt.
Planner	`min_speed_mps`	0.20	Untergrenze der Fahrgeschwindigkeit.
Planner	`stop_speed_mps`	0.00	Geschwindigkeit im Stoppfall.
Planner	`stop_clearance_m`	0.25	Sicherheitsabstand für Stop.
Planner	`caution_clearance_m`	0.50	Abstand für Wiederfreigabe.
Planner	`steering_slowdown_start_rad`	0.40	Start der Geschwindigkeitsreduktion bei starkem Lenken.

Node	**Parameter**	**Wert**	**Einfluss**
Planner	`steering_-slowdown_-full_rad`	0.70	Maximale Reduktion bei großem Lenkwinkel.
Planner	`heading_smoothing_alpha`	0.4	Glättet Richtungswechsel zwischen Scans.
Planner	`speed_smoothing_alpha`	0.3	Glättet Beschleunigung (Bremsen direkt).
Control	`input_timeout_sec`	0.50	Sicherheitsstopp bei veralteten Planner-Daten.
Control	`angle_to_-steering_gain`	-1.00	Skaliert/invertiert Lenkwinkel auf Lenkung.
Control	`max_steering_angle_-rad`	0.45	Hardware-Limit des Lenkwinkels.
Control	`min_speed_mps`	0.18	Minimale Ausgabegeschwindigkeit.
Control	`max_speed_mps`	1.80	Maximale Ausgabegeschwindigkeit.

Neben den Laufzeitparametern verwendet der C++-Kern `follow_the_gap_v0` zusätzlich interne, nicht zur Laufzeit parametrierbare Geometrie- und Gewichtungskonstanten. Diese sind in Tabelle 2.2 zusammengefasst, da sie die Passierbarkeit von Lücken und damit das resultierende Fahrverhalten ebenfalls wesentlich beeinflussen.

Tabelle 2.2: Interne FTG-Konstanten des C++-Kerns `follow_the_gap_v0`.

Konstante	**Wert**	**Bedeutung**
`kCarRadius`	0.40 m	Effektiver Sicherheitsradius zur Hindernis-Aufblähung und damit maßgeblich für die als passierbar bewertete Gap-Breite.
`kTurnRadius`	0.30 m	Modellannahme für die Kurvenfähigkeit des Fahrzeugs.
`kTrackMinWidth`	0.35 m	Mindestbreite für bestimmte Corner- und Gap-Bewertungen.
`kDistanceToCorner`	0.22 m	Sicherheitsabstand bei Corner-bezogenem Verhalten.
`kGapWeightCoefficient`	100.0	Gewichtung der Gap-Richtung in der Zielrichtungsbestimmung.
`kCornerWeightCoefficient`	100.0	Gewichtung corner-spezifischer Anteile in der Zielwahl.

2.7.4 Messmethodik und Datenerfassung

Zur experimentellen Bewertung des Systems wurden sämtliche relevanten Systemgrößen während der Fahrt zeitlich synchron aufgezeichnet und anschließend offline ausgewertet. Die Datenerfassung erfolgte mit `rosbag2`, sodass sowohl Rohdaten als auch Zwischen- und Ausgangsgrößen des gesamten FTG-Stacks reproduzierbar analysiert werden konnten.

Die Auswertung fokussierte sich auf vier zentrale Beobachtungsgrößen:

- **Distanzverlauf:** Charakterisierung der Umgebungssituation über den gefilterten Scan sowie den minimalen Frontabstand,
- **Heading-Winkel:** Bewertung der vom FTG-Kern bestimmten Zielrichtung und deren Weiterverarbeitung im Planner,
- **Geschwindigkeit:** Analyse der Sollgeschwindigkeit des Planners und der tatsächlich ausgegebenen Ackermann-Geschwindigkeit,
- **Gap-Erkennung:** Bewertung, ob und wie stabil freie, befahrbare Lücken detektiert wurden.

Zur Online-Beobachtung des Systemzustands wurde zusätzlich *Foxglove Studio* eingesetzt. Die Live-Visualisierung diente insbesondere der Plausibilitätsprüfung von Scan,

Heading, Gap-Erkennung, Planner-Zuständen, Ackermann-Ausgabe und Visualisierungstopics während der Testdurchführung.

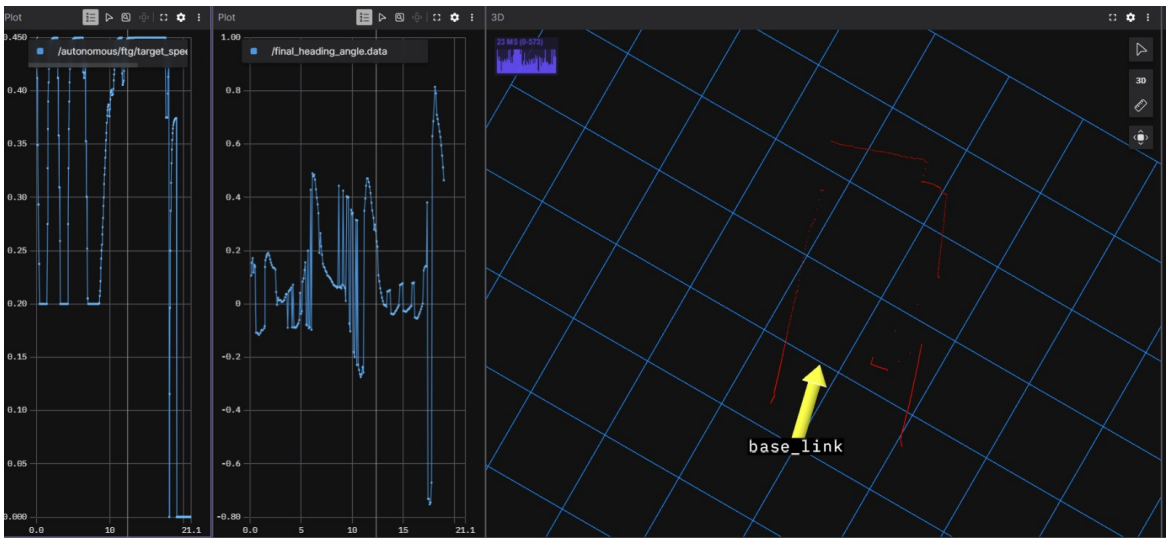

Abbildung 2.5: Exemplarisches Foxglove-Layout zur Online-Beobachtung des FTG-Stacks während der Validierungsfahrten. Dargestellt sind Sensorik, Visualisierungstopics und zentrale Zustandsgrößen des Planungs- und Steuerpfads.

Die eigentliche quantitative Bewertung erfolgte auf Basis der aufgezeichneten Bags in einer anschließenden Offline-Analyse. Hierzu wurden die Signalverläufe der Validierungsläufe zeitlich ausgewertet und in Form von Metriken und Zeitreihenplots aufbereitet. Zu den wesentlichen Kenngrößen zählten unter anderem:

- mittlerer und minimaler Frontabstand,
- mittlere Sollgeschwindigkeit und Zeitanteil mit Stopp-Anforderung,
- Zeitanteil mit erfolgreicher Gap-Erkennung,
- Auftreten von Sättigungen im Heading- bzw. Gap-Winkel,
- Auftreten von Planner-Zuständen wie *normal*, *stale* oder *no valid gap found*,
- qualitative Bewertung von Ausweichmanövern, Passagewahl und Eckdurchfahrt.

Die daraus resultierenden Validierungsplots, Vergleichstabellen und Anforderungsabgleiche werden in Kapitel 3 zur Diskussion und Bewertung der Teststrecken herangezogen.

3 Diskussion und Bewertung

Dieses Kapitel bewertet die entwickelte Systemarchitektur hinsichtlich ihrer technischen Eignung, Robustheit, Reaktionsfähigkeit und Übertragbarkeit. Im Mittelpunkt stehen dabei nicht nur die grundsätzliche Funktionsfähigkeit des scan-basierten Follow-the-Gap-Stacks, sondern auch die Frage, wie zuverlässig Hindernisse bzw. freie befahrbare Bereiche erkannt werden, wie sicher die resultierenden Ausweichmanöver ausfallen und wie stark die Parametrierung das Fahrverhalten beeinflusst.

3.1 Bewertung der gewählten Gesamtarchitektur

Die Entscheidung für eine strikt scan-basierte Pipeline stellt eine bewusste Abkehr von komplexeren, mehrstufigen Wahrnehmungsansätzen dar. Anstelle einer expliziten Objektextraktion oder einer modellbasierten Hindernisrepräsentation wird der freie und belegte Raum direkt aus dem LiDAR-Signal interpretiert. Dies entspricht dem Grundgedanken des Follow-the-Gap-Verfahrens, das auf eine reaktive Auswahl befahrbarer Lücken aus dem aktuellen Sichtfeld abzielt.

Durch den direkten Übergang von `/scan` zu einer gefilterten Scan-Repräsentation wird auf zusätzliche Abstraktionsebenen wie Objektdetektion, Segmentierung oder Kartierung verzichtet. Dies reduziert:

- die Anzahl potenzieller Fehlerquellen innerhalb der Verarbeitungskette,
- den Parametrierungs- und Integrationsaufwand,
- die Latenz zwischen Sensormessung und Fahrreaktion.

Gerade für reaktive Verfahren wie Follow-the-Gap ist diese Vereinfachung fachlich sinnvoll, da der Algorithmus explizit für rohnahe Sensordaten formuliert wurde und seine Stärke in der schnellen lokalen Reaktion auf Hindernisse liegt.

3.2 Analyse der Planner- und Control-Schicht

Die Trennung zwischen algorithmischem Kern, Planner und Control erzeugt eine klare funktionale Hierarchie innerhalb des Systems. Der FTG-Kern liefert eine fahrzeugagnostische Richtungsinformation, während Planner und Control diese Information in fahrbare und sicherheitsgerichtete Sollwerte überführen.

Die gewählte Aufteilung ermöglicht:

- Wiederverwendbarkeit des FTG-Kerns unabhängig von der Fahrzeugplattform,
- unabhängige Anpassung der Fahrstrategie durch Parametrierung der Planner-Logik,
- klare Definition von Verantwortlichkeiten innerhalb der Pipeline.

Insbesondere fungiert der Planner als **Policy-Schicht**, die sicherstellt, dass die reaktive Planung in physikalisch realisierbare und sichere Fahrbefehle überführt wird. Die Control-Schicht bildet anschließend die Schnittstelle zur bestehenden Fahrzeugsteuerung und begrenzt die Planner-Ausgabe entsprechend der fahrzeugspezifischen Randbedingungen.

3.3 Bewertung der experimentellen Ergebnisse

Die durchgeführten Fahrversuche zeigen, dass das System in strukturierten Innenräumen grundsätzlich funktionsfähig ist, die erreichte Qualität des Fahrverhaltens jedoch deutlich von Umgebungsgeometrie und Parametrierung abhängt. Besonders relevant sind dabei die Stabilität der Gap-Erkennung, das Auftreten von Stopps bzw. Fast-Stop-Zuständen, die Qualität der Eckdurchfahrt sowie der Einfluss des Sichtfelds und der geometrischen FTG-Konstanten.

3.3.1 Quantitativer Vergleich der Validierungsläufe

Tabelle 3.1 fasst die wichtigsten Kenngrößen der beiden Validierungsläufe zusammen. Die Werte basieren auf der Offline-Auswertung der aufgezeichneten Bags und beziehen sich sowohl auf die wahrnehmungsnahen Größen als auch auf die vom Planner und von der Control-Schicht erzeugten Signale.

Tabelle 3.1: Vergleich zentraler Metriken der beiden Validierungsläufe.

Metrik	**Validierungslauf Test 1**	**Validierungslauf Test 2**
Gap-Erkennung	89,8 %	56,0 %
Planner-Zustand *normal*	90,0 %	55,5 %
Planner-Zustand *stale*	0,0 %	32,8 %
Planner-Zustand *no valid gap found*	10,0 %	11,7 %
Mittlere-Sollgeschwindigkeit `target_speed`	0,335 m/s	0,226 m/s
Mittlerer-Frontabstand `front_clearance`	0,601 m	0,685 m
Heading-Sättigung	11,4 %	–
Gap-Sättigung	5,2 %	–
Stopp-Anforderung	gering	44,2 %
Kollisionen	0	0

Die Gegenüberstellung zeigt, dass Validierungslauf Test 1 insgesamt ein deutlich stabileres Verhalten aufweist. Lauf Test 2 ist dagegen durch einen wesentlich höheren Anteil kritischer Zustände gekennzeichnet, insbesondere durch häufige Planner-Zustände *stale* sowie einen hohen Anteil von Stopp-Anforderungen.

3.3.2 Zeitreihenbasierte Detailanalyse

Zur qualitativen Interpretation der numerischen Kennwerte wurden die wichtigsten Signalverläufe beider Fahrten zeitlich ausgewertet. Dabei wurden insbesondere Frontabstand, Heading-Winkel, Gap-Winkel, Sollgeschwindigkeit und Planner-Zustände herangezogen.

Abbildung 3.1: Zeitreihenbasierte Auswertung von Validierungslauf Test 1. Dargestellt sind Frontabstand, Heading- und Gap-Winkel, Sollgeschwindigkeit sowie Zustandsgrößen des Planners.

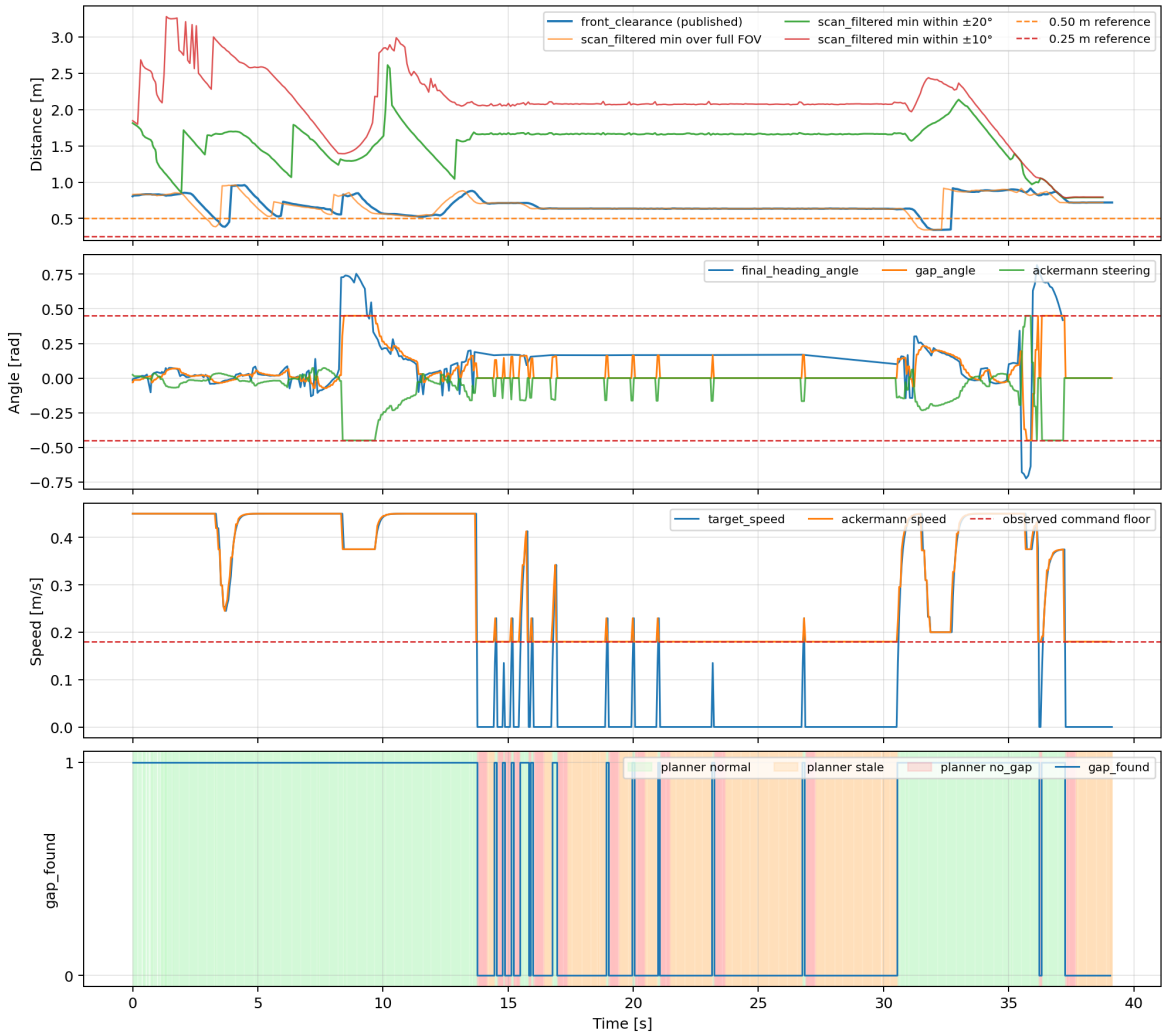

Abbildung 3.2: Zeitreihenbasierte Auswertung von Validierungslauf Test 2. Dargestellt sind Frontabstand, Heading- und Gap-Winkel, Sollgeschwindigkeit sowie Zustandsgrößen des Planners.

Die Zeitreihen zeigen, dass Lauf Test 1 über weite Strecken stabile und konsistente Reaktionen des Systems aufweist. In Lauf Test 2 treten dagegen deutlich häufiger Phasen auf, in denen keine valide Lücke erkannt wird oder der Planner auf veraltete Eingangsdaten reagiert. Dies führt zu einer größeren Zahl unterbrochener oder verlangsamter Fahrabschnitte.

3.3.3 Bewertung der Hinderniserkennung

Die Hinderniserkennung erfolgt im vorliegenden System nicht als semantische Objekterkennung, sondern als Interpretation freier und belegter Bereiche im LiDAR-Scan. Bewertet wird daher nicht die Klassifikation einzelner Objekte, sondern die Zuverlässigkeit, mit der der Stack befahrbare Lücken erkennt und den unmittelbaren Fahrweg als frei oder eingeschränkt bewertet.

Die Ergebnisse zeigen:

- Die Gap-Erkennung ist in Validierungslauf A mit knapp 90 % sehr stabil.
- In Validierungslauf B sinkt die Gap-Erkennungsrate deutlich auf rund 56 %.
- Der Unterschied ist nicht primär auf geringere Frontabstände zurückzuführen, sondern auf eine instabilere Interpretation der Umgebungssituation.

Dies ist ein wichtiges Ergebnis: Eine hohe mittlere Clearance allein garantiert noch kein robustes Fahrverhalten. Entscheidend ist, ob aus der gemessenen Umgebung eine konsistente und fahrbare Lücke abgeleitet werden kann.

3.3.4 Bewertung der Ausweichmanöver

Die Qualität der Ausweichmanöver wurde anhand folgender Kriterien bewertet:

- Kollisionsfreiheit,
- erfolgreiche Passage der Hindernisse,
- Qualität der Seitenwahl an asymmetrischen Engstellen,
- Eckdurchfahrt,
- Kontinuität des Fahrverlaufs.

Beide Läufe verliefen kollisionsfrei. Dies ist ein zentraler Nachweis dafür, dass das System grundsätzlich in der Lage ist, auf statische Hindernisse sicher zu reagieren. Allerdings zeigt sich ein deutlicher Unterschied in der Qualität der Manöver:

- **Validierungslauf A:** Das Fahrzeug umfuhr alle Hindernisse und meisterte die 90°-Ecke grundsätzlich erfolgreich. Der Fahrverlauf ist als überwiegend stabil und funktional zu bewerten.
- **Validierungslauf B:** Die Reaktion blieb zwar kollisionsfrei, war jedoch deutlich defensiver und weniger stabil. Häufige Stopps bzw. Fast-Stop-Zustände und eine geringere Verfügbarkeit der Gap-Erkennung weisen auf eine eingeschränkte Robustheit in dieser Konfiguration hin.

Damit ist der prinzipielle Nachweis der Reaktionsstrategie erbracht, jedoch nicht in gleicher Güte für beide Szenarien.

3.3.5 Einfluss der Parametrierung auf das Fahrverhalten

Ein zentrales Ergebnis der Validierung ist, dass die Parametrierung des Systems den resultierenden Fahrstil unmittelbar beeinflusst. Dieser Zusammenhang betrifft sowohl die Wahrnehmungsstufe als auch Planner und Control sowie die internen FTG-Konstanten.

Einfluss des Sichtfelds

Das Frontsichtfeld bestimmt, wie breit die Umgebung in die lokale Reaktionsentscheidung einfließt. Ein zu großes Sichtfeld erhöht die Sensitivität gegenüber seitlichen Wänden und kann in engen Korridoren dazu führen, dass eigentlich passierbare Situationen als kritisch interpretiert werden. Ein reduziertes Sichtfeld führt dagegen zu einer robusteren Vorwärtsorientierung, kann jedoch seitliche Hindernisse später in die Entscheidungsfindung einbeziehen.

Einfluss der Clearance-Schwellen

Die Parameter `stop_clearance_m` und `caution_clearance_m` bestimmen, wie defensiv das Fahrzeug auf Hindernisannäherungen reagiert. Höhere Schwellen führen zu früherem Bremsen und damit zu einem konservativen Fahrstil. Niedrigere Schwellen erhöhen die Flussfähigkeit, können aber die Sicherheitsreserve verringern.

Einfluss der Steering-Slowdown-Parameter

Die Parameter `steering_slowdown_start_rad` und `steering_slowdown_full_rad` bestimmen, wie stark Kurven- bzw. Ausweichwinkel auf die Sollgeschwindigkeit wirken. Wird die maximale Geschwindigkeitsreduktion zu früh erreicht, kann dies insbesondere in Ecksituationen zu Corner-Blockaden oder unnötigen Stopps führen.

Einfluss der internen FTG-Geometrie

Besonders relevant ist der interne Parameter `kCarRadius`. Er bestimmt über die Hindernis-Aufblähung, welche Lücken als passierbar betrachtet werden. Der derzeit verwendete Wert von 0,4 m ist für ein Fahrzeug mit etwa 0,30 m Breite stark konservativ und führt dazu, dass enge, real befahrbare Passagen im Algorithmus als blockiert erscheinen.

Insgesamt zeigt sich: Die Parametrierung beeinflusst nicht nur die numerischen Signale des Systems, sondern den gesamten resultierenden Fahrstil, also etwa ob das Fahrzeug eher flüssig, defensiv, ruckartig oder blockierend reagiert.

3.3.6 Empfehlungen aus der Validierung

Aus den Messergebnissen ergeben sich mehrere konkrete Optimierungsempfehlungen:

1. **Reduktion des effektiven Fahrzeugradius im FTG-Kern:** Der Parameter `kCarRadius` sollte von 0,4 m auf etwa 0,20 m reduziert bzw. künftig als konfigurierbarer ROS-Parameter umgesetzt werden.

2. **Trennung von FTG-Sichtfeld und Clearance-Sichtfeld:** Für die Berechnung von `front_clearance` sollte ein engeres Sichtfenster verwendet werden als für den eigentlichen FTG-Scan, um Seitenwände in Korridoren weniger stark zu gewichten.

3. **Überprüfung der Stop-Propagation:** Stopp-Anforderungen des Planners sollten in der Control-Stufe konsequent bis zum Ackermann-Ausgang durchgereicht werden.

4. **Further Tuning der Planner-Policy:** Die Clearance- und Steering-Schwellen sollten so gewählt werden, dass Corner-Blockaden vermieden werden, ohne die Sicherheitsreserve unangemessen zu reduzieren.

3.3.7 Grenzfälle und Grenzen des Systems

Trotz der positiven Ergebnisse treten typische Einschränkungen des gewählten Ansatzes auf:

- keine strategische Entscheidung in Sackgassen oder global komplexen Szenarien,
- potenziell suboptimale Wahl in asymmetrischen oder mehrdeutigen Geometrien,
- starke Abhängigkeit von Sichtfeld und Clearance-Parametrierung,
- konservatives Verhalten bei übergroß gewählter Fahrzeuggeometrie,
- mögliche Diskrepanzen zwischen Planner-Stop und tatsächlich ausgegebenem Fahrbefehl.

Diese Grenzen sind teils algorithmisch inhärent und teils durch die konkrete Implementierung und Parametrierung bedingt. Sie bilden damit zugleich die zentralen Ansatzpunkte für zukünftige Weiterentwicklungen.

4 Fazit und Ausblick

4.1 Fazit

Ziel dieser Projektarbeit war die Entwicklung, Integration und Validierung einer ROS 2-basierten Hinderniserkennung mit geeigneter Reaktionsstrategie auf dem MxCarkit-Modellfahrzeug. Im Mittelpunkt stand dabei nicht allein die Implementierung eines einzelnen Algorithmus, sondern der Aufbau einer vollständigen, reproduzierbaren Verarbeitungskette von der LiDAR-Sensorik bis zur fahrbaren Ackermann-Ausgabe.

Die entwickelte Kernarchitektur lautet:

`/scan` → `scan_preprocessor_node` → `/autonomous/ftg/scan_filtered` → `follow_the_gap_v0` → `ftg_planner_node` → `ftg_command_node` → `/autonomous/ackermann_cmd`

Damit wurde ein durchgängiger, modularer und nachvollziehbarer Datenpfad realisiert, der sich in die bestehende MXCK-Steuerarchitektur einfügt und die vorhandene Sicherheits- und Moduslogik nicht umgeht.

Die Validierung zeigt, dass das System unter Laborbedingungen grundsätzlich funktionsfähig ist. In beiden Validierungsläufen konnten Hindernisse kollisionsfrei umfahren und die Reaktionsstrategie des Systems experimentell nachgewiesen werden. Insbesondere die erfolgreiche Integration von Perception, FTG-Kern, Planner und Control belegt, dass ein scan-basiertes Hindernisvermeidungsverfahren auf dem MxCarkit praktisch betrieben werden kann.

Gleichzeitig hat die Auswertung deutlich gemacht, dass die Qualität des Fahrverhaltens wesentlich von der Parametrierung abhängt. Sichtfeld, Clearance-Schwellen, Steering-Slowdown und die internen geometrischen FTG-Konstanten beeinflussen unmittelbar, ob das Fahrzeug eher flüssig, defensiv, konservativ oder instabil reagiert. Die Arbeit zeigt damit nicht nur, dass der Stack prinzipiell funktioniert, sondern auch, dass reproduzierbare Validierung und kritische Parameterauswertung unverzichtbar für eine belastbare Bewertung sind.

Die Ergebnisse lassen sich wie folgt zusammenfassen:

- Die scan-basierte Hauptpipeline wurde konsistent in die bestehende MXCK-Architektur integriert.
- Die Rollen von Perception, FTG-Kern, Planner und Control sind funktional sauber getrennt.

- Die experimentellen Fahrversuche belegen die grundsätzliche Eignung des Systems für strukturierte Innenraumszenarien mit statischen Hindernissen.
- Die Robustheit ist jedoch nicht in allen Szenarien gleichermaßen hoch und hängt stark von Parametrierung und Umgebungsgeometrie ab.
- Die Arbeit identifiziert damit nicht nur eine funktionierende Lösung, sondern auch konkrete technische Grenzen und Optimierungsbedarfe.

Zusammenfassend wurden die wesentlichen Ziele der Aufgabenstellung erreicht:

- fachliche Einarbeitung in Hinderniserkennung, ROS 2 und MXCK-Systemarchitektur,
- modularer Entwurf und Integration eines scan-basierten Hindernisvermeidungsstacks,
- experimentelle Validierung anhand realer Fahrversuche,
- sowie kritische Analyse von Leistungsfähigkeit, Robustheit und Grenzen.

Die Arbeit leistet damit einen Beitrag zur systematischen Integration und Bewertung eines reaktiven Hindernisvermeidungsverfahrens auf einer realen ROS 2-Fahrzeugplattform.

4.2 Ausblick

Aus den Ergebnissen ergeben sich mehrere konkrete Weiterentwicklungsrichtungen.

Parametrierbare FTG-Geometrie

Die derzeit noch im C++-Kern fest kodierten Geometrieparameter, insbesondere `kCarRadius`, sollten künftig als ROS-Parameter verfügbar gemacht werden. Dies würde eine systematische Anpassung an reale Fahrzeugabmessungen ermöglichen und die experimentelle Vergleichbarkeit verschiedener Konfigurationen verbessern.

Verbesserung der Clearance-Bewertung

Für enge Korridorszenarien erscheint es sinnvoll, das für die Geschwindigkeitsbewertung genutzte Clearance-Fenster vom eigentlichen FTG-Sichtfeld zu trennen. Dadurch ließe sich der Einfluss seitlicher Wände auf die Längsdynamik verringern, ohne die Hinderniswahrnehmung des FTG-Kerns selbst einzuschränken.

Konsistentere Stop-Umsetzung

Ein weiterer wichtiger Entwicklungsschritt ist die konsequente Durchreichung von Planner-Stopps bis zur Control-Stufe und damit bis zum Ackermann-Ausgang. Gerade für die Sicherheitsvalidierung ist entscheidend, dass erkannte kritische Situationen nicht nur im Planner, sondern auch im endgültigen Fahrbefehl vollständig umgesetzt werden.

Erweiterte Validierung

Die vorliegende Arbeit bewertet das System in strukturierten Innenraumszenarien mit statischen Hindernissen. Für eine weitergehende Aussagekraft wären zusätzliche Untersuchungen sinnvoll, beispielsweise:

- systematische Parameterstudien mit mehreren Konfigurationssätzen,
- Vergleich verschiedener Sichtfelder und Clearance-Strategien,
- Validierung mit dynamischen Hindernissen,
- sowie Auswertung weiterer Fahrszenarien wie Engstellen, Sackgassen oder stärker variierende Kurvengeometrien.

Weiterentwicklung über lokale Reaktion hinaus

Langfristig kann der Stack durch zusätzliche Komponenten ergänzt werden, etwa durch globale Pfadplanung, Lokalisierung oder sensorfusionierte Umfeldwahrnehmung. Damit ließe sich die derzeit rein lokale Reaktionsstrategie um eine strategische Navigation erweitern.

Schlussbemerkung

Die wichtigste Erkenntnis dieser Arbeit ist, dass ein robuster Hindernisvermeidungsstack nicht allein durch den zugrunde liegenden Algorithmus bestimmt wird. Entscheidend ist vielmehr das Zusammenspiel aus Sensorvorverarbeitung, geometrischer Konsistenz, parametrierter Fahrstrategie, sauberer Systemintegration und nachvollziehbarer Validierung. Erst diese Gesamtheit macht aus einem theoretisch bekannten Verfahren eine praktisch nutzbare Fahrfunktion auf einer realen ROS 2-Plattform.

Literaturverzeichnis

[1] Sezer, V.; Gokasan, M.: *A Novel Obstacle Avoidance Algorithm: "Follow the Gap Method"*. Robotics and Autonomous Systems, Vol. 60, No. 9, 2012, pp. 1123–1134. DOI: 10.1016/j.robot.2012.05.021.

[2] Endler, M.: *Using ROS 2 for High-Speed Maneuvering in Autonomous Driving*. Bachelor's Thesis, Czech Technical University in Prague, 2022.

[3] Czech Technical University – Intelligent and Interactive Graphics Group (CTU-IIG): *F1TENTH Follow-the-Gap Implementation (f1t-ftg)*. Available at: `https://github.com/CTU-IIG/f1t-ftg` (Accessed: 2026-04).

[4] O'Kelly, M. et al.: *F1TENTH: An Open-Source Evaluation Environment for Continuous Control and Reinforcement Learning*. NeurIPS Workshop on Machine Learning for Autonomous Driving, 2019.

[5] Open Robotics: *ROS 2 Documentation*. Available at: `https://docs.ros.org` (Accessed: 2026-04).

[6] Gemini 3 Flash et al.: *KI-generierte Foto*. Erstellt am 12. April 2026. Software: ChatGPT und Google AI.

[7] Elalami, O. et al.: *ftg_mxck – ROS 2 Follow-the-Gap Stack for MXCarkit*. GitHub Repository. Available at: `https://github.com/OmarMElalami/ftg_mxck` (Accessed: 2026-04).

[8] MXCK Platform: *mxck2_ws – ROS 2 Workspace for MXCarkit*. Available at: `https://github.com/william-mx/mxck2_ws` (Accessed: 2026-04).